

Documentation

Test Unitaire

Rédacteurs :

Gaël TUCZAPSKI

Date : 13 janv 2025 - Version V01

SOMMAIRE

SOMMAIRE.....	2
TABLES DES MODIFICATIONS.....	3
1. CADRE GENERAL DU PRODUIT.....	3
1.1. Titre du produit.....	3
1.2. Objectif.....	3
Documentation du Projet.....	3
1. Présentation Générale.....	3
0. my_alpha_number_t(nbr).....	3
1. sum(a, b).....	4
2. my_size_alpha_t(str = ").....	4
3. my_display_alpha_t().....	4
4. my_array_alpha_t(str).....	4
5. my_is_posi_neg_t(nbr).....	4
6. fibo(n).....	5
7. my_display_alpha_reverse_t().....	5
8. my_length_array_t(arr).....	5
9. my_display_unicode_t(arr).....	5
10. quickSort(arr).....	6
11. tspBrutForce(distances).....	6
11.1 permuter(arr).....	6
12. resoudreSudoku(grille).....	6
12.1 estValide(grille, ligne, col, num).....	7
5. Tests Unitaires.....	7
5.1 Lancement des tests.....	7

Documentation du Projet

1. Présentation Générale

Ce projet (**tp-tests-unitaires**) a pour objectif de montrer comment réaliser des **tests unitaires** avec Jest pour plusieurs fonctions JavaScript. Il inclut :

- Un **fichier source** `index.js` (dans le dossier `src`) qui contient 13 fonctions (indexées de 0 à 12).
- Un **fichier de test** `index.class.test.js` (dans le dossier `tests`), qui contient l'ensemble des tests unitaires avec Jest.

L'exécution des tests se fait via la commande `npm run test`, qui génère également un rapport de couverture de code.

0. `my_alpha_number_t(nbr)`

- **But** : Convertir n'importe quelle valeur en chaîne de caractères.
- **Paramètre** :
 - `nbr` : n'importe quelle valeur (souvent un `number` ou un `string`).
- **Retour** : Une chaîne de caractères.

Exemple d'utilisation :

```
my_alpha_number_t(123);    // Retourne "123"
my_alpha_number_t('Hello'); // Retourne "Hello"
my_alpha_number_t();       // Retourne "undefined"
```

1. `sum(a, b)`

- **But** : Additionner deux nombres.
- **Paramètres** :
 - `a` : nombre à additionner (type `number`)
 - `b` : nombre à additionner (type `number`)
- **Retour** :
 - La somme `a + b` si `a` et `b` sont des nombres.
 - Retourne `0` dans tous les autres cas.

Exemple d'utilisation :

```
sum(2, 3);    // 5 sum('abc', 10); // 0 (car 'abc' n'est pas un number)
```

2. `my_size_alpha_t(str = '')`

- **But** : Retourner la taille (longueur) d'une chaîne de caractères, en utilisant une boucle `while`.
- **Paramètre** :
 - `str` : une chaîne de caractères (par défaut `' '` si non fourni).

- **Retour** : Nombre de caractères dans `str`.

Exemple d'utilisation :

```
my_size_alpha_t('Hello'); // 5 my_size_alpha_t(); // 0
```

3. `my_display_alpha_t()`

- **But** : Retourner l'alphabet en minuscule de `a` à `z`.
- **Paramètres** : Aucun.
- **Retour** : Une chaîne `'abcdefghijklmnopqrstuvwxyz'`.

Exemple d'utilisation :

```
my_display_alpha_t(); // "abcdefghijklmnopqrstuvwxyz"
```

4. `my_array_alpha_t(str)`

- **But** : Transformer une chaîne de caractères en tableau de caractères, en utilisant `my_size_alpha_t` pour déterminer la longueur.
- **Paramètre** :
 - `str` : chaîne de caractères.
- **Retour** : Un tableau contenant chaque caractère de `str`.

Exemple d'utilisation :

```
my_array_alpha_t('Hello'); // ['H', 'e', 'l', 'l', 'o']
```

5. `my_is_posi_neg_t(nbr)`

- **But** : Vérifier si un nombre est positif ou négatif (ou zéro).
- **Paramètre** :
 - `nbr` : un nombre.
- **Retour** :
 - `'NEGATIVE'` si `nbr <= 0`.
 - `'POSITIF'` si `nbr > 0`.

Exemple d'utilisation :

```
my_is_posi_neg_t(5); // "POSITIF" my_is_posi_neg_t(-3); // "NEGATIVE"
my_is_posi_neg_t(0); // "NEGATIVE"
```

6. `fibo(n)`

- **But** : Calculer la valeur de la suite de Fibonacci pour un entier `n`, en utilisant une **réursion**.

- **Paramètre :**
 - `n` : un nombre entier.
- **Retour :**
 - 0 si `n <= 0`.
 - 1 si `n == 1` ou `n == 2`.
 - `fibo(n - 1) + fibo(n - 2)` sinon.

Exemple d'utilisation :

```
fibo(5); // 5 (suite: 1, 1, 2, 3, 5) fibo(-1); // 0
```

7. my_display_alpha_reverse_t()

- **But :** Retourner l'alphabet en minuscule de `z` à `a`.
- **Paramètres :** Aucun.
- **Retour :** `'zyxwvutsrqponmlkjihgfedcba'`.

Exemple d'utilisation :

```
my_display_alpha_reverse_t(); // "zyxwvutsrqponmlkjihgfedcba"
```

8. my_length_array_t(arr)

- **But :** Retourner la longueur d'un tableau, sans utiliser `arr.length`.
- **Paramètre :**
 - `arr` : un tableau (ex. `[1,2,3]`).
- **Retour :** Nombre d'éléments dans `arr`.

Exemple d'utilisation :

```
my_length_array_t([1,2,3]); // 3 my_length_array_t([]); // 0
```

9. my_display_unicode_t(arr)

- **But :** Convertir un tableau de codes ASCII en une chaîne de caractères. Ne traite que certains codes (lettres, chiffres, espace).
- **Paramètre :**
 - `arr` : un tableau de nombres, représentant des codes ASCII.
- **Retour :** Chaîne de caractères formée par la concaténation des codes reconnus.

Exemple d'utilisation :

```
my_display_unicode_t([72,101,108,108,111,32,49,50]); // "Hello 12"
(72='H', 101='e', 108='l', 32=' ', 49='1', 50='2')
```

10. quickSort(arr)

- **But** : Trier un tableau de nombres en utilisant l'**algorithme Quick Sort**.
- **Paramètre** :
 - **arr** : un tableau de nombres.
- **Retour** : Un **nouveau tableau** (ou le même) trié par ordre croissant.

Exemple d'utilisation :

```
quickSort([3,1,2]); // [1,2,3] quickSort([5,3,8,1,2]); // [1,2,3,5,8]
```

11. tspBrutForce(distances)

- **But** : Résoudre le **problème du voyageur de commerce** (TSP) par **force brute**, en parcourant toutes les permutations de villes.
- **Paramètre** :
 - **distances** : un objet contenant les distances entre les villes, sous forme de clé/valeur, par exemple :

```
{ A: { A: 0, B: 10, C: 15 }, B: { A: 10, B: 0, C: 20 }, C: { A: 15, B: 20, C: 0 } }
```

- **Retour** : Un objet { **minDistance**, **meilleurePermutation** } où :
 - **minDistance** : la distance totale la plus courte.
 - **meilleurePermutation** : le chemin (ordre des villes) le plus optimal.

Exemple d'utilisation :

```
const distances = { A: { A: 0, B: 10, C: 15 }, B: { A: 10, B: 0, C: 20 }, C: { A: 15, B: 20, C: 0 } }; tspBrutForce(distances); // { minDistance: 35, meilleurePermutation: ['A','B','C'] }
```

Cette fonction s'appuie sur la fonction **permuter(arr)** (voir ci-dessous) pour générer toutes les permutations des villes.

11.1 permuter(arr)

- **But** : Générer toutes les permutations d'un tableau. (Fonction auxiliaire pour le TSP.)
- **Paramètre** :
 - **arr** : un tableau, par exemple ['A', 'B', 'C'].
- **Retour** : Un tableau de tableaux, où chaque sous-tableau est une permutation de **arr**.

Exemple d'utilisation :

```
permuter(['A','B','C']); // Retour possible : [ [ 'A', 'B', 'C' ], [ 'A', 'C', 'B' ], [ 'B', 'A', 'C' ], ... ]
```

12. resoudreSudoku(grille)

- **But** : Résoudre un **Sudoku** en utilisant une approche **backtracking**.
- **Paramètre** :

- **grille** : un tableau 2D (9x9) représentant les cases d'un sudoku, où 0 indique une case vide.
- **Retour** :
 - **true** si le sudoku a été résolu, et la grille est alors modifiée in-place.
 - **false** sinon (cas de grille impossible).

Exemple d'utilisation :

```
let grille = [ [5,3,0, 0,7,0, 0,0,0], [6,0,0, 1,9,5, 0,0,0], [0,9,8,
0,0,0, 0,6,0], ... ]; resoudreSudoku(grille); // grille est alors
complétée, renvoie true si succès
```

12.1 estValide(grille, ligne, col, num)

- **But** : Vérifier si l'on peut placer **num** dans la case **[ligne][col]** sans violer les règles du sudoku.
- **Paramètres** :
 - **grille** : le sudoku sous forme de tableau 2D.
 - **ligne** : index de ligne.
 - **col** : index de colonne.
 - **num** : nombre à tester (1 à 9).
- **Retour** :
 - **true** si **num** peut être placé sans conflit.
 - **false** sinon.

Cette fonction est utilisée en interne par **resoudreSudoku**.

5. Tests Unitaires

5.1 Lancement des tests

Pour exécuter l'ensemble des tests unitaires, lancez la commande :

```
npm run test
```

Contact : gael.tuczapsky@etu.mines-ales.fr